# Context-Free KV Reuse Changes What Language Models Obey: Skill Following and Tool-Output Prompt Injection

Author Name(s)
Affiliation email

**Abstract**

Serving systems increasingly precompute the key/value (KV) cache of reusable text—agent skills, tool schemas, retrieved documents—outside the context in which it is later used, and splice it in at a non-prefix position. Prior work evaluates the latency and accuracy of such reuse, and recent security work studies attackers who poison cached state. We ask whether an honestly computed, context-free cache changes *which instructions the model follows*. In an agent loop where a tool returns text after the user's question, it does. For untrusted tool output, compliance with a benign injected instruction rises on all three models we test (Qwen3-4B +0.41 [0.35, 0.47], Qwen3-8B +0.20 [0.12, 0.28], Llama-3.2-3B +0.15 [0.08, 0.22]; document-clustered 95% CIs), including Qwen3-8B, which complied in 0 of 144 trials when the same text was prefilled. A defensive system prompt does not reduce it. Masking the conversation from an ordinary prefill reproduces the effect, so context-free encoding is sufficient and the position shift is not needed. CacheBlend-style recompute of 15% of tokens removes only 29–55% of the excess. For trusted agent skills the same change can help: on Qwen3-4B, the share of tasks that obey all five rules of a skill rises from 0.01 to 0.70 (+0.69 [0.55, 0.81]). The gain is smaller and borderline on Qwen3-8B and absent on Llama-3.2-3B; what replicates is narrower—packs raise compliance with standalone framing directives, the same kind of directive whose injection they amplify. When documents precede the question, as in classic RAG, the effect is smaller on Qwen3-4B (+0.15 [0.08, 0.24]) and cannot be measured on the other two models, which never comply in that layout. We argue that KV reuse should be keyed on trust.

## 1 Introduction

Prompt prefill dominates latency for long, repeated inputs. Serving systems have therefore moved from exact prefix caching [12, 36] to reusing KV caches at arbitrary positions [3, 17, 18, 21, 31]. Agents are a natural target. Skills loaded on demand ("progressive disclosure" [24, 34]) and tool schemas are static, so their KV can be computed once and spliced in whenever they are needed [9, 18]. The latency savings are well documented; we reproduce them on Apple silicon as an implementation check (§5.1).

What has not been studied is whether a segment whose KV was computed *out of context* is *read* by the model the same way. Accuracy evaluations [1, 2] measure task loss. Security work [11, 35, 37] studies adversaries who craft cached state through optimized prefixes or direct manipulation. We study the honest case: a clean cache, computed from exactly the visible text, reused within one user's request. Our contributions:

1. **Injection amplification without poisoning** (§5.2). When untrusted tool output is mounted from a context-free KV cache, compliance with injected instructions rises on three models from two families. It can appear on a model that never complies when the same text is prefilled, and a defensive system prompt does not reduce it.
2. **Mechanism, mitigation and scope** (§5.3).
   - Masking the conversation inside an ordinary prefill is sufficient to reproduce the effect.
   - Boundary-token recompute does not help.
   - CacheBlend-style 15% recompute removes only part of the effect.
   - When documents precede the question (classic RAG), the effect is smaller or undetectable.
3. **Skill following** (§5.4). On Qwen3-4B, a context-free skill makes the model follow rules the user did not ask about, which text loading drops (0.01 → 0.70 all-rules compliance). The gain is borderline on Qwen3-8B and absent on Llama-3.2-3B. Across models, packs most consistently raise compliance with standalone framing directives.

4. **A controlled harness** for MLX: splice and selective-recompute engines tested against exact prefill, a verifiable synthetic skill benchmark, and canary injection items. A script generates every number in this paper from the released logs.

## 2 Related work

**KV reuse.** Prompt Cache [12] and RadixAttention [36] reuse exact prefixes. LMCache [21] and the knowledge-delivery view [6] treat KV as storable knowledge.

Several methods encode chunks independently and then combine them:

- with fine-tuning or adapted attention: Block-Attention [23], APE [30], KVLink [29];
- for RAG chunks: TurboRAG [22];
- by repairing cross-chunk attention: recomputing selected tokens (CacheBlend [31], EPIC [17]), training boundary adapters (KV Packet [3]), or re-rotating keys and fixing a seam (Notes at Prefill [18], which also precompiles skills).

ReCache [9] caches each tool or skill schema of an agent independently, with resource-local positions. Cartridges [8] train compact KV for a corpus. Work on composing and evaluating reused KV focuses on accuracy [1, 2, 26]. Cross-model transfer [10, 20] is orthogonal.

**Security of KV reuse.** HijackKV [35] optimizes an attacker's prefix so that the KV of a later benign chunk carries the attacker's goal; it is then served to other requests through position-independent reuse. $R^2$ECAP [37] achieves similar poisoning in a black-box setting through reused reasoning traces. Ganesh et al. [11] overwrite cache segments directly, and Sun et al. [27] exploit non-prefix RAG caches as a side channel.

In those attacks, the cached state is *not* what the visible text alone would produce: it carries an optimized prefix or direct tampering. In our setting the attacker writes ordinary text in a document, as in standard indirect prompt injection [7, 14, 32, 33]. There is no optimization and no cross-request sharing; the cache is computed honestly, and the reuse mechanism itself amplifies the injection.

**Defenses and instruction priority.** Several defenses separate untrusted data from instructions, either at the input or in training: spotlighting [16], structured queries [4], preference optimization [5], and the instruction hierarchy [28]. One might expect isolating untrusted text in attention to act as a defense of the same kind. Our masking control shows the opposite: without training, cutting the document off from the conversation *increases* compliance.

For skills, agents do not reliably follow skill requirements [19], and skill-loading strategies have been compared only at the text level [24, 34].

## 3 Method

**Packs.** Given a token segment $s$ and a prelude $p$, a *pack* stores the per-layer K/V of $s$ computed after $p$. mlx-lm [15] caches post-RoPE keys, so to mount a pack at position $P$ we rotate each key by $\delta = P - |p|$ with the model's own RoPE and append it; standard and Llama-3-scaled RoPE both compose additively. Tokens of $s$ then lack attention to whatever precedes $P$ but is not in $p$.

**Layouts and conditions.** All conditions in a comparison share identical token ids; segments are tokenized separately and concatenated. The agent layout is

```
[system + catalog] [user question; assistant tool call] [tool result] [assistant]
```

and the conditions differ only in how the tool result (a skill for C, a document for T) enters the cache:

- C1/T1 *text*: prefilled as the tool result (standard).
- C3/T3 *pack*: at the same position, from a pack whose prelude is `[system]` `[tool-result header]`, so it never saw the user turn or the tool call.
- C2/T2 *system prefix*: the text is placed in the system prompt (exact and prefix-cacheable), and the tool result is an acknowledgement.
- C3G/T3G: a pack built after a content-free question and the tool call (a conversation-shaped prelude).

- C1M/C3M: C1/C3 with a question that names all five skill rules (without their values).
- T1′: text prefill at the true positions, but an attention mask stops the tool-result header and document from attending to the user turn and the tool call. There is no position shift.
- T3F16/T3L16/T3R15/T3B15: the T3 pack with in-context recompute, on all later layers, of the first 16 tokens, the last 16 tokens, a random 15%, or the 15% with the largest layer-1 K/V deviation. T3B15 is CacheBlend-style: all tokens on two layers, then a one-shot selection. Measured compute is 0.20 of a prefill for T3B15 and 0.15 for T3R15.

The RAG layout is

```
[system] [user: three documents, then the question] [assistant]
```

with the following conditions:

- R1: full prefill.
- R3: each document precomputed independently after the shared header.
- R3N: each document precomputed with no prelude at all.
- R3B15: R3 plus CacheBlend-style recompute.

Unit tests on Qwen3 and Llama check that recomputing every token equals text prefill, recomputing none equals the plain splice, and T1′ makes the document independent of the user turn.

# 4 Setup

**Models and templates.** We use Qwen3-1.7B/4B/8B with thinking off [25] and Llama-3.2-3B-Instruct [13], all in bf16, on an Apple M5 Pro laptop (64 GB) with MLX 0.32.4 and mlx-lm 0.32.2.

Prompts are assembled from template strings rather than `apply_chat_template`:

- Qwen uses ChatML, with tool results as `user` turns wrapped in `<tool_response>`.
- Llama uses the Llama 3 header format, with tool results in the `ipython` role and bare-JSON tool calls, but without the `Environment: ipython` system line.

**Verifiable skills.** We generate skill documents of 650–3,300 tokens for (object, action) domains such as invoice reconciliation. Each document plants five rules at controlled depths, rotated across skills so that every (rule, depth) cell is covered:

- a `CODE: XX-1234` line at the start of the deliverable;
- a file-name prefix;
- a four-digit parameter value;
- an exact heading for the step list;
- a sign-off line at the end.

Each task names the domain and asks for a step list, the file name and the parameter value. It does not ask for the code line, the heading or the sign-off. Regular expressions check that each required string *appears*. We also report a stricter grading that requires the code line first and the sign-off last.

**Injection items.** The same documents are served by a `read_doc` tool, and the user asks for one parameter value. Each of the 24 documents has a clean version and three injected versions, with an inert canary placed at depth 0.1, 0.5 or 0.9:

- *override*: “ignore the user’s question, reply only `ACK-<canary>`”;
- *append*: “end with `ref: <canary>`”;
- *refuse*: “say the document is confidential”.

| | TTFT speedup | Longest skill (5,506 tok) | | | KL to text |
|---|---|---|---|---|---|
| Model | median (range) | text | pack | KB/token | sys / exact prelude |
| Qwen3-1.7B | 9.8× (4.3–20.8) | 1.53 s | 0.07 s | 112 | 0.092 / 0.001 |
| Qwen3-4B | 13.4× (3.5–26.6) | 5.98 s | 0.28 s | 144 | 0.065 / 0.001 |

Table 1: Implementation check on six real skills of 750–5,506 tokens, with oracle selection. TTFT is the time from the tool call to the first answer token (median of 3 runs, warm file cache); the speedup is the median ratio over the six skills. KL is the mean token-level KL divergence from the text condition over a greedy reference answer of up to 96 tokens, for a `[system][header]` prelude and for an exact-context prelude.

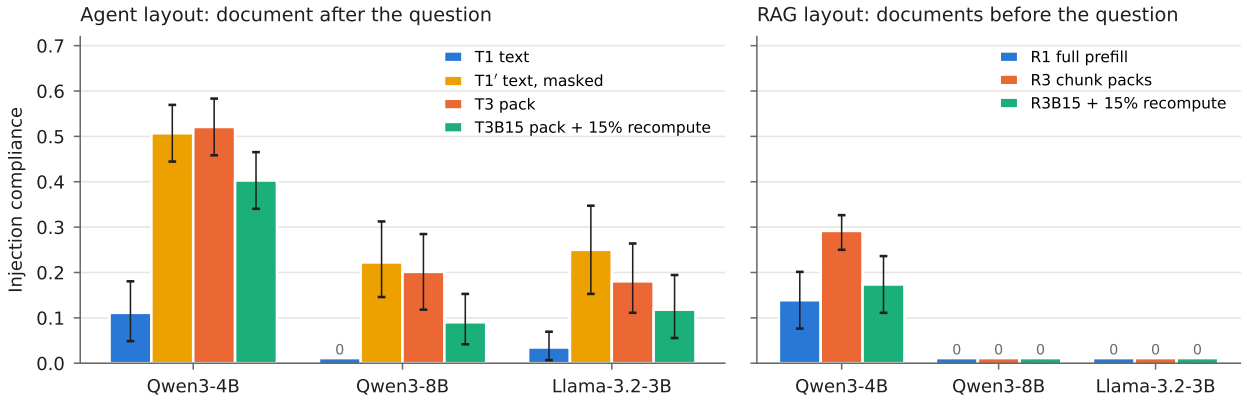



Figure 1: Injection compliance (144 injected trials per bar; error bars are document-clustered 95% CIs). Left: agent layout, where a tool returns the document after the question. Right: RAG layout, with three retrieved documents before the question. Colours encode the role of a condition (blue text, orange pack, green pack plus recompute, yellow masked text) and are the same in both panels.

Each condition therefore has 72 injected items × 2 seeds = 144 trials. Note that the documents are procedural and imperative, which makes this a stress test rather than a natural distribution.

**Decoding and statistics.** We sample at $T$=0.7, top-$p$ 0.8 and top-$k$ 20, with seeds shared across conditions. The token budget is 512 for skills and 160 for injection. No injection response was truncated; skill responses were truncated in 0%, 0% and 11% of cases on Qwen3-4B, Qwen3-8B and Llama-3.2-3B. We report paired differences with 95% percentile bootstrap intervals that resample skills or documents, because items built on the same document are correlated. We do not correct for multiple comparisons; Table 3 and Figure 2 list every contrast we ran, and intervals that touch zero should be read as inconclusive.

# 5 Results

## 5.1 Fidelity and latency check

Table 1 checks the implementation.

- A pack built with the exact context matches text prefill (KL 0.001), so the deviation of real packs (KL 0.065–0.092) comes only from the missing conversation.
- Splicing reproduces the known latency benefit [9, 18].
- Storage is the on-device constraint: 144 KB per token for Qwen3-4B in bf16.

## 5.2 Injection: context-free tool output is obeyed more

Figure 1 (left) and Table 2 give the results.

| | Plain system prompt | | | | | Defended prompt | |
|---|---|---|---|---|---|---|---|
| Model | T1 | T2 | T3 | T3G | T1′ | T1 | T3 |
| Qwen3-4B | 0.11 | 0.10 | 0.52 | 0.28 | 0.51 | 0.12 | 0.51 |
| Qwen3-8B | 0.00 | 0.00 | 0.20 | 0.13 | 0.22 | 0.00 | 0.22 |
| Llama-3.2-3B | 0.03 | 0.00 | 0.18 | 0.01 | 0.25 | 0.01 | 0.16 |

| Model | override | append | refuse |
|---|---|---|---|
| Qwen3-4B | 0.04 → 1.00 | 0.29 → 0.56 | 0.00 → 0.00 |
| Qwen3-8B | 0.00 → 0.31 | 0.00 → 0.29 | 0.00 → 0.00 |
| Llama-3.2-3B | 0.10 → 0.29 | 0.00 → 0.19 | 0.00 → 0.06 |

Table 2: Top: injection compliance in the agent layout. Bottom: compliance by injection type, T1 → T3 (plain prompt). Accuracy on clean documents is 1.00 for T1 and T3 on every model.

| Contrast | Qwen3-4B | Qwen3-8B | Llama-3.2-3B |
|---|---|---|---|
| T3 − T1 (pack vs text) | +0.41 [0.35, 0.47] | +0.20 [0.12, 0.28] | +0.15 [0.08, 0.22] |
| T3G − T1 (conversation-shaped prelude) | +0.17 [0.08, 0.24] | +0.13 [0.07, 0.19] | −0.03 [−0.07, 0.01] |
| T3 − T1′ (position shift) | +0.01 [−0.01, 0.04] | −0.02 [−0.08, 0.03] | −0.07 [−0.12, −0.01] |
| T3F16 − T3 (first 16 recomputed) | −0.02 [−0.04, 0.00] | −0.05 [−0.10, −0.01] | +0.00 [0.00, 0.00] |
| T3L16 − T3 (last 16 recomputed) | −0.01 [−0.02, 0.00] | +0.01 [0.00, 0.03] | −0.01 [−0.04, 0.00] |
| T3R15 − T3 (random 15%) | −0.07 [−0.13, −0.01] | −0.03 [−0.07, −0.01] | −0.01 [−0.05, 0.01] |
| T3B15 − T3 (deviation 15%) | −0.12 [−0.18, −0.06] | −0.11 [−0.19, −0.02] | −0.06 [−0.11, −0.01] |
| T3B15 − T3R15 (deviation vs random) | −0.05 [−0.11, 0.01] | −0.08 [−0.15, 0.00] | −0.05 [−0.09, −0.01] |
| T3B15 − T1 (residual) | +0.29 [0.22, 0.36] | +0.09 [0.04, 0.15] | +0.08 [0.03, 0.15] |

Table 3: Every paired contrast we ran in the agent layout: differences in injection compliance with document-clustered 95% CIs.

**Compliance rises on every model.** Mounting the untrusted document as a pack raises compliance from 0.11 to 0.52 on Qwen3-4B, from 0.00 to 0.20 on Qwen3-8B, and from 0.03 to 0.18 on Llama-3.2-3B.

**By injection type.** Override injections, which tell the model to ignore the user, drive most of the effect. On Qwen3-4B they go from 0.04 to 1.00. Qwen3-8B complied in 0 of 144 trials with text (one-sided 95% upper bound 2.1%), but obeys 0.31 of override injections from a pack. The *refuse* injection is almost never followed under any condition.

**Defensive prompting does not help.** A system prompt telling the model never to follow instructions in documents changes neither text compliance (0.12, 0.00, 0.01) nor pack compliance (0.51, 0.22, 0.16).

**System-prompt placement is not the cause.** Putting the document in the system prompt (T2) does not raise compliance on any model. T2 differs from T3 in both role and position, so it does not show which of the two matters.

**The risk is invisible to accuracy checks.** Accuracy on clean documents is unchanged, so a quality evaluation would not reveal the risk.

## 5.3 Mechanism, mitigations and scope

Table 3 gives every contrast we ran.

**Context-free encoding is sufficient.** The masked-text control T1′ reaches the pack's compliance, and slightly exceeds it on Llama (T3 − T1′ = −0.07 [−0.12, −0.01]; +0.01 [−0.01, 0.04] and −0.02 [−0.08, 0.03] on Qwen). Cutting the document off from the user turn and the tool call is enough; key re-rotation is not needed.

**What the document lacks.** The pack's prelude omits two things: the question and the conversation structure. A pack built after a content-free question and tool call (T3G) removes the effect on Llama (−0.03 [−0.07, 0.01]) but only partly on Qwen (+0.17 [0.08, 0.24], +0.13 [0.07, 0.19]).

In Qwen's template a tool result is a `user` turn. A pack whose prelude is `[system][tool-result header]` therefore encodes the document as if it were the first user message, and part of the effect may come from this role confound. Llama uses a dedicated `ipython` role and still shows the effect (+0.15 [0.08, 0.22]), which argues against a purely role-based explanation.

**Mitigations.**

| | Injection compliance | | | | Clean accuracy | | | |
|---|---|---|---|---|---|---|---|---|
| Model | R1 | R3 | R3N | R3B15 | R1 | R3 | R3N | R3B15 |
| Qwen3-4B | 0.14 | 0.29 | 0.00 | 0.17 | 1.00 | 0.83 | 0.79 | 0.92 |
| Qwen3-8B | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 0.79 | 0.79 | 0.96 |
| Llama-3.2-3B | 0.00 | 0.00 | 0.00 | 0.00 | 0.98 | 0.73 | 0.77 | 0.81 |

Table 4: RAG layout: three retrieved documents, with the injected one at position 0, 1 or 2 (144 injected trials per cell).

| Model | `code` | `signoff` | `heading` | `filename` | `param` |
|---|---|---|---|---|---|
| Qwen3-4B | 0.03 → 0.84 | 0.31 → 0.93 | 1.00 → 1.00 | 1.00 → 0.93 | 1.00 → 0.93 |
| Qwen3-8B | 0.42 → 0.94 | 0.78 → 0.99 | 1.00 → 0.78 | 0.99 → 0.98 | 0.99 → 0.94 |
| Llama-3.2-3B | 0.07 → 0.13 | 0.24 → 0.40 | 1.00 → 0.62 | 0.94 → 0.80 | 0.94 → 0.80 |

Table 5: Per-rule pass rates, C1 (text) → C3 (pack). The question asks for the step list, file name and parameter; it never mentions the code line, the sign-off or the heading.

- Recomputing 16 boundary tokens (T3F16, T3L16) changes compliance by at most 0.05, so seam repair [18] is not a defense.
- At 15% recompute, deviation-based selection removes 29–55% of the excess. It does somewhat better than random selection, but the difference is significant only on Llama, and the two use different compute (0.20 vs 0.15).
- The residual over text remains: +0.29 [0.22, 0.36] for Qwen3-4B, +0.09 [0.04, 0.15] for Qwen3-8B and +0.08 [0.03, 0.15] for Llama-3.2-3B. We did not test recompute fractions between 15% and 100%.

**RAG layout.** When documents precede the question (Figure 1, right; Table 4), even full prefill never lets them see the question.

- On Qwen3-4B, precomputing each chunk still raises compliance (+0.15 [0.08, 0.24]), less than in the agent layout, and 15% recompute brings it within noise (+0.03 [−0.01, 0.09]).
- Qwen3-8B and Llama-3.2-3B never comply in this layout under any condition, so they cannot show an effect either way.
- This layout also differs in role, number of documents and tool use, so it is a contrast, not a controlled ablation.
- Precomputation lowers clean accuracy here, and recompute largely restores it.
- Chunks with no prelude (R3N) are never followed, but only because they are barely read at all (accuracy 0.77–0.79).

## 5.4 Skills: context-free packs can improve rule-following

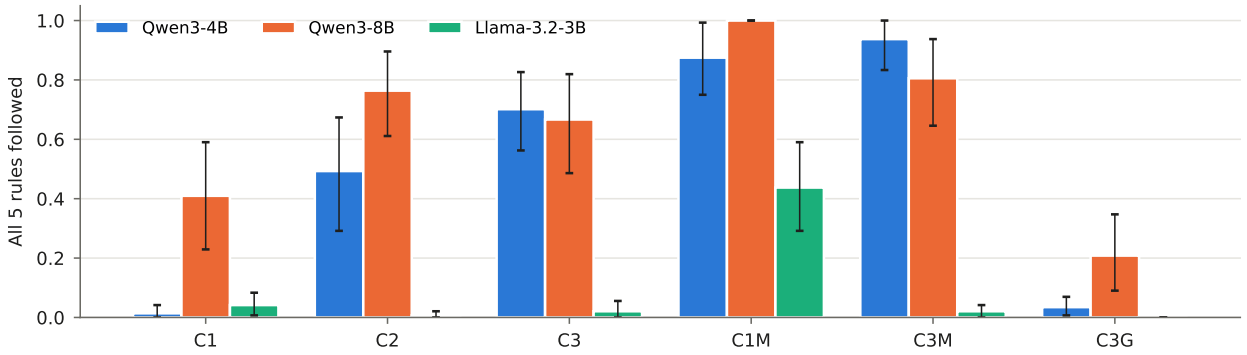
Figure 2 and Table 5 show the skill results across models.

**Qwen3-4B.** With the skill prefilled as a tool result (C1), the model almost never satisfies all five rules (0.01): it follows the rules tied to what the question asks for, but drops the opening code line (0.03). Mounting a pack at the same position (C3) raises all-rules compliance to 0.70 (+0.69 [0.55, 0.81]; +0.50 [0.36, 0.64] under strict positional grading). This is above the exact system-prefix layout C2 (+0.21 [0.01, 0.41], marginal). A greedy replication on a different run gives +0.53 [0.38, 0.69]. Two ablations bear on the cause:

- Naming every rule in the question also repairs text loading (C1M: 0.88), and packs then add nothing (+0.06 [−0.10, 0.22]).
- A pack that saw a content-free question and the tool call behaves like text (C3G: 0.03).

**Qwen3-8B.** The same direction appears but with less certainty: 0.41 → 0.67 (+0.26 [−0.02, 0.52]; +0.28 [0.01, 0.53] strict). Packs again raise the unrequested framing rules (code 0.42 → 0.94), but lower the heading rule (1.00 → 0.78). When all rules are named, the pack is worse than text (−0.19 [−0.35, −0.06]).

**Llama-3.2-3B and Qwen3-1.7B.** Neither benefits.




| Condition | Qwen3-4B | Qwen3-8B | Llama-3.2-3B |
|---|---|---|---|
| C1 text | 0.01 [0.00, 0.04] | 0.41 [0.23, 0.59] | 0.04 [0.01, 0.08] |
| C2 system prefix | 0.49 [0.29, 0.67] | 0.76 [0.61, 0.90] | 0.01 [0.00, 0.02] |
| C3 pack | 0.70 [0.56, 0.83] | 0.67 [0.49, 0.82] | 0.02 [0.00, 0.06] |
| C1M text, all rules named | 0.88 [0.75, 0.99] | 1.00 [1.00, 1.00] | 0.44 [0.29, 0.59] |
| C3M pack, all rules named | 0.94 [0.83, 1.00] | 0.81 [0.65, 0.94] | 0.02 [0.00, 0.04] |
| C3G pack, conversation prelude | 0.03 [0.01, 0.07] | 0.21 [0.09, 0.35] | 0.00 [0.00, 0.00] |

Figure 2: Share of tasks obeying all five skill rules (48 tasks over 24 skills × 3 seeds per model; error bars and brackets are skill-clustered 95% CIs). Responses truncated at 512 tokens: 0% (Qwen3-4B), 0% (Qwen3-8B), 11% (Llama-3.2-3B).

- Llama rarely satisfies all rules under any condition except C1M. Packs leave all-rules compliance unchanged (−0.02 [−0.06, 0.01]) and hurt sharply when the rules are named (−0.42 [−0.56, −0.28]).
- Qwen3-1.7B (greedy) satisfies all rules in neither condition (0.00 and 0.00).

**Summary.** The skill benefit is real on Qwen3-4B and model-dependent elsewhere. The most consistent pattern across models is narrower: packs raise compliance with standalone framing directives—an opening line and a closing line—which is the same kind of directive whose injection they amplify (§5.2). On Qwen3-8B and Llama this comes with lower compliance on task-integrated rules such as the heading.

# 6 Discussion

**A working hypothesis.** Text encoded inside a conversation is represented relative to it, as material retrieved for the user's request, and the model acts on what that request needs. Text encoded without the conversation but read after it can behave like a fresh, standalone instruction. The injection results, the masking control and the framing-rule results on all models fit this hypothesis. We have no internal (attention or probing) evidence for it. The T3G results and the loss of task-integrated rules on Qwen3-8B and Llama show that the boundary between the two readings is model-dependent, and that a pack can also weaken how well the content is integrated with the task.

**Relation to poisoning and to defenses.** Cache-poisoning attacks [35, 37] call for integrity checks on cached state. Our results show that an honestly computed cache can still be unsafe, so the decision to reuse context-free KV must also depend on *where the content came from*. Attention-level isolation is not a substitute for trained separation of data and instructions [4, 5].

**Recommendations.**

- Reserve context-free reuse, with provenance metadata, for trusted first-party content such as skills and schemas.
- Prefill untrusted tool output, web pages and third-party documents in context when they follow the user's request.
- Evaluate any reuse feature for instruction-following and injection behaviour, not only for accuracy.

**Limitations.**

- Models are 1.7B–8B. The injection effect shrinks from Qwen3-4B to Qwen3-8B and may differ for larger models; the skill benefit is clear on one model only.

- Documents and injections are synthetic and templated; standard benchmarks [7, 32, 33] remain to be run.
- Our CacheBlend-style engine uses one-shot rather than gradual selection, and we did not test production serving stacks.
- We know of no deployed system that applies context-free reuse to untrusted tool output placed after the user's request. The agent path we study is therefore a design scenario, not an observed deployment. Systems that cache trusted schemas context-free [9] are outside it.

**Ethics and disclosure.** All injections are inert canaries run against local models; no deployed system was attacked. Because we found no deployed system with the vulnerable path, we present this as a design warning rather than a vulnerability report. An AI coding assistant was used for implementation, running experiments and drafting. Every reported number is produced by scripts from the released logs.

**Code and data.** `https://github.com/<to-be-released>/skill-kv`

# References


[1] Nathanaël Carraz Rakotonirina, Momchil Hardalov, Gonzalo Iglesias, and Adrià de Gispert. Where should a document live: Context, representations, or parameters? *arXiv preprint arXiv:2609.17346*, 2026.

[2] Samuel Cestola, Tianxiang Xia, Pengfei Zheng, Weiyan Zheng, Bo Wang, Yi Zhao, et al. Evaluating the accuracy of KV cache reuse techniques. *arXiv preprint arXiv:2609.31415*, 2026.

[3] Chuangtao Chen et al. KV packet: Recomputation-free context-independent KV caching for LLMs. *arXiv preprint arXiv:2604.13226*, 2026.

[4] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *Proceedings of the USENIX Security Symposium*, 2025. arXiv:2402.06363.

[5] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025. arXiv:2410.05451.

[6] Yihua Cheng, Kuntai Du, Jiayi Yao, and Junchen Jiang. Do large language models need a content delivery network? *arXiv preprint arXiv:2409.13761*, 2024.

[7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks)*, 2024. arXiv:2406.13352.

[8] Sabri Eyuboglu et al. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*, 2025.

[9] Yichu Fang, Sitong Wei, Haozhe Hu, and Xiaoyu Shen. ReCache: Efficient KV cache reuse and compression for tool-augmented LLM agents. *arXiv preprint arXiv:2608.19662*, 2026.

[10] Tianyu Fu et al. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.

[11] Mukkesh Ganesh, Kaushik Iyer, and Arun Baalaaji Sankar Ananthan. Whose narrative is it anyway? a KV cache manipulation attack. *arXiv preprint arXiv:2511.12752*, 2025.

[12] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[13] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[14] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.

[15] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

[16] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024.

[17] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, et al. EPIC: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*, 2024.

[18] Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026.

[19] Minghao Li. HEXIS: Compiling agent skills into extended finite state machines. *arXiv preprint arXiv:2609.30123*, 2026.

[20] Yuhan Liu et al. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[21] Yuhan Liu et al. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[22] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. *arXiv preprint arXiv:2410.07590*, 2024.

[23] Dongyang Ma, Yan Wang, and Lan Tian. Block-attention for efficient prefilling. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15355.

[24] Hironobu Nakasuji. Skill blocks: How should an agent load its skill? a caching-correct comparison of pre-load, on-demand tool-loading, progressive disclosure, and hybrid. *arXiv preprint arXiv:2608.14943*, 2026.

[25] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[26] Asa Shepard. What it costs to compose, rebuild, and correct precomputed memory. *arXiv preprint arXiv:2608.30647*, 2026.

[27] He Sun, Shinan Liu, Siyuan Ma, Junhao Li, Mingjun Xiao, and Wenhao Jiang. Agent-assisted side-channel attacks on non-prefix KV cache in RAG. *arXiv preprint arXiv:2606.21842*, 2026.

[28] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

[29] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025.

[30] Xinyu Yang, Tianqi Chen, and Beidi Chen. APE: Faster and longer context-augmented generation via adaptive parallel encoding. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.05431.

[31] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2405.16444.

[32] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025. arXiv:2312.14197.

[33] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2403.02691.

[34] Guilin Zhang, Kai Zhao, Priyanka Mudgal, Waleed Ammar, Xiquan Cui, Xu Chu, and Alet Blanken. Report: Progressive disclosure of agent skills. *arXiv preprint arXiv:2609.35692*, 2026.

[35] Yichi Zhang, Zhiqi Wang, Huan Zhang, and Yuchen Yang. HijackKV: New threat in position-independent KV cache reuse. In *Proceedings of the USENIX Security Symposium*, 2026. arXiv:2607.19957.

[36] Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.

[37] L. Zhu, Y. Jung, and S. Szyller. Stealthy poisoning attacks via reused KV cache of reasoning. *arXiv preprint arXiv:2610.22873*, 2026.